PRÁCTICA 4: ORGANIZANDO NUESTRA BIBLIOTECA MUSICAL. TAD ÁRBOL

CONTEXTO DEL PROBLEMA

Las plataformas de música no solo recomiendan canciones, sino que también organizan grandes bibliotecas musicales para permitir búsquedas rápidas y eficientes.

En esta práctica ampliaréis vuestro explorador musical incorporando una nueva funcionalidad: la **organización de canciones mediante un árbol binario de búsqueda**. Esto permitirá no solo encontrar canciones rápidamente, sino también generar playlists ordenadas y realizar búsquedas avanzadas siguiendo distintos criterios (por ejemplo, identificador, artista o duración).

Para ello implementaréis el **TAD Árbol Binario de Búsqueda (BST – Binary Search Tree)**, una de las estructuras ampliamente utilizadas en bases de datos, sistemas de archivos y motores de búsqueda. Esto permitirá:

- Localizar canciones rápidamente
- Mostrar la biblioteca ordenada
- Explorar diferentes formas de recorrido

OBJETIVOS

- Implementar un **TAD Árbol binario de búsqueda (BST)**.
- Aprender a manejar recursión.
- Insertar canciones (Music) en el árbol siguiendo un criterio de orden.
- Buscar canciones de forma eficiente.
- Comprender la diferencia entre estructuras lineales (pilas, colas, listas), relacionales (grafos) y jerárquicas (árboles).

NORMAS

Los programas que se entreguen deben seguir los criterios indicados en el Moodle de prácticas, considerando, además:

- Estar escritos en lenguaje C, cumpliendo el estándar C90.
- Compilar sin errores ni *warnings* usando las opciones `-Wall` y `-pedantic`.
- Incorporar un adecuado control de errores (pensar tanto en casos base como extremos).
- Gestionar correctamente la memoria dinámica (**no debe haber fugas de memoria**).
- Respetar la **organización y nomenclatura de ficheros y prototipos** indicados en este enunciado.

PLANIFICACIÓN

- **Semana 1:** Ejercicio 1
- **Semana 2:** Ejercicio 2
- **Semana 3:** Ejercicio 3 y 4
- **Semana 4:** Ejercicio 4 y/u opcional **+ entrega**

La entrega final se realizará a través de Moodle, siendo la fecha límite en evaluación continua **el 07 de mayo** hasta el final del día. El fichero comprimido deberá seguir el formato `Px_apellido1_nombre1_apellido2_nombre2`, siendo `x` el número de pareja. Este fichero deberá contener, al menos, la siguiente organización de ficheros:

```
--- Px_apellido1_nombre1_.../
    |--- music.c
    |--- music.h
    |--- radio.c
    |--- radio.h
    |--- types.h
    |--- bstree.c
    |--- bstree.h
    |--- p4_e1.c
    |--- p4_e2.c
    |--- p4_e3.c
    |--- Makefile
    |--- data_music_10.txt
    |--- data_music_100.txt
    |--- data_music_1K.txt
```

EJERCICIO 1. MIN, MAX, CONTAINS, INSERT

En este ejercicio se implementarán e incluirán las rutinas `tree_find_min`, `tree_find_max`, `tree_contains` y `tree_insert` dentro de una implementación parcial proporcionada del TAD Árbol Binario de Búsqueda (BST).

DEFINICIÓN DE LA ESTRUCTURA DE DATOS E IMPLEMENTACIÓN DE PRIMITIVAS

En el fichero `bstree.h` se proporciona la interfaz del TAD Árbol y en `bstree.c` se tiene una implementación casi completa del mismo. La estructura de datos que se usa para implementar el TAD Árbol BST es la siguiente:

```
struct _BSTree {
  BSTNode *root;
  P_ele_print print_ele;
  P_ele_cmp cmp_ele;
};
```

Básicamente, se tiene **dos punteros a funciones** que servirán para imprimir y comparar los elementos del árbol, que se pasarán como argumentos al inicializar el árbol. Además, se tiene el **nodo raíz**.

Los nodos del árbol son del tipo privado `_BSTNode`, que está formado por un puntero al dato que almacena y dos punteros a sus nodos hijos, izquierdo y derecho, que serán NULL si no existen. La estructura empleada es:

```
typedef struct _BSTNode {
  void *info;
  struct _BSTNode *left;
  struct _BSTNode *right;
} BSTNode;
```

Las rutinas que implementar en `bstree.c`, **de forma recursiva**, son:

- `tree_find_min`: Devuelve el elemento más pequeño almacenado conforme a la rutina de comparación del árbol.
- `tree_find_max`: Devuelve el elemento más grande almacenado conforme a la rutina de comparación del árbol.
- `tree_contains`: Comprueba si un elemento existe en el árbol.
- `tree_insert`: Inserta un nuevo elemento en el árbol (si el elemento ya se encuentra en el árbol no se debe insertar, pero la función devolverá OK).

**Si has usado un for o un while en alguna rutina del árbol…
el árbol se ha enfadado contigo.**

PRUEBA DE FUNCIONAMIENTO BÁSICO

Se proporciona el programa principal `p4_e1.c` que incluye unas pruebas básicas sobre las rutinas implementadas. Su ejecución se estructura de la siguiente forma:

```
./p4_e1 [fichero_in] [id] [mode]
```

Siendo [fichero_in] un fichero de lectura de los datos de entrada (se proporcionan varios de distinto tamaño), [id] un identificador de canción y [mode] con los valores posibles 'normal' o 'sorted'.

Analiza el código proporcionado para entender qué operaciones se realizan y qué primitivas del TAD se están utilizando, prestando especial atención a la opción que ordena o no los datos previamente a través de la función `qsort`. Realiza pruebas usando distintos ficheros de entrada (que pueden tener elementos repetidos o no) y comprueba que los tiempos de creación y búsqueda varían según si el programa se ejecuta en modo normal o en modo ordenado. **P1** ¿Por qué es así?, ¿hay alguna propiedad del árbol que permita explicar este comportamiento? **Responde a estas preguntas en un comentario al final de p4_e1.c**

Salida esperada (orientativa – los tiempos pueden variar):

```
./p4_e1 data_music_10 8 normal
```

```
Mode: normal
Tree building time: 3 ticks (0.000003 seconds)
Tree size: 10
Tree depth: 6
Min element in tree: [1, Bohemian Rhapsody, Queen, 354, 0, 3] - 1 ticks (0.000001 seconds)
Max element in tree: [10, Shape of You, Ed Sheeran, 233, 0, 2] - 0 ticks (0.000000 seconds)
Element found - 0 ticks (0.000000 seconds)
```

```
./p4_e1 data_music_10 8 sorted
```

```
Mode: sorted
Tree building time: 4 ticks (0.000004 seconds)
Tree size: 10
Tree depth: 4
Min element in tree: [1, Bohemian Rhapsody, Queen, 354, 0, 3] - 1 ticks (0.000001 seconds)
Max element in tree: [10, Shape of You, Ed Sheeran, 233, 0, 2] - 1 ticks (0.000001 seconds)
Element found - 0 ticks (0.000000 seconds)
```

EJERCICIO 2. REMOVE

En este ejercicio implementaremos la primitiva pública `tree_remove` para eliminar un elemento del árbol, cuya declaración se encuentra en `bstree.h`. Para esto, se necesitaba implementar además una función privada recursiva `_bst_remove_rec`, con el siguiente prototipo:

```
BSTNode *_bst_remove_rec(BSTNode *pn, const void *elem, P_ele_cmp cmp_elem)
```

ALGORITMO

A continuación, se proporciona un pseudocódigo para la función privada recursiva `_bst_remove_rec`:

En el TAD Árbol, recuerda probar casos extremos, como árboles vacíos, nodos con sólo un hijo, ramas izquierdas, derechas, etc.

Algorithm 1: _bst_remove_rec : Recursive remove function for Binary Search Tree

```
input  : Pointer to current tree node: pn
         Pointer to element: elem
         Pointer to compare function: cmp_elem
output: Pointer to node that remains in current position
if pn is NULL then
    return NULL
end
// Compare elem with current node info
if elem is smaller than pn.info then
    // Look in the left subtree
    pn.left = _bst_remove_rec(pn.left, elem, cmp_elem)
end
else if elem is greater than pn.info then
    // Look in the right subtree
    pn.right = _bst_remove_rec(pn.right, elem, cmp_elem)
end
else if elem is equal to pn.info then
    // We found the element to remove
    if pn has no children then
        _bst_node_free(pn)
        return NULL
    end
    else if pn has only right child then
        ret_node = pn.right
        _bst_node_free(pn)
        return ret_node
    end
    else if pn has only left child then
        ret_node = pn.left
        _bst_node_free(pn)
        return ret_node
    end
    else if pn has both children then
        // We remove pn and replace it with the smaller element in the right subtree (which is
          removed from there)
        aux_node = _bst_find_min_rec(pn.right)
        pn.info = aux_node.info
        pn.right = _bst_remove_rec(pn.right, aux_node.info, cmp_elem)
        return pn
    end
end
return pn
```

PRUEBA DE FUNCIONAMIENTO BÁSICO

Utiliza nuevamente el programa `p4_e1.c`, descomentando la funcionalidad relativa a la eliminación de un elemento. Es **muy importante** que profundices en los distintos casos de eliminación que se contemplan en el algoritmo. Crea los ficheros de pruebas que consideres oportunos y recuerda que puedes utilizar tipos simples de datos (int, string, etc.).

Salida esperada (extendida del ejercicio anterior):

```
./p4_e1 data_music_10 8 normal
```

```
Mode: normal
Tree building time: 3 ticks (0.000003 seconds)
Tree size: 10
Tree depth: 6
Min element in tree: [1, Bohemian Rhapsody, Queen, 354, 0, 3] - 1 ticks (0.000001 seconds)
Max element in tree: [10, Shape of You, Ed Sheeran, 233, 0, 2] - 0 ticks (0.000000 seconds)
Element found - 0 ticks (0.000000 seconds)
Removing element in tree: OK - 1 ticks (0.000001 seconds)
Tree size: 9
Tree depth: 6
```

```
./p4_e1 data_music_10 8 sorted
```

```
Mode: sorted
Tree building time: 4 ticks (0.000004 seconds)
Tree size: 10
Tree depth: 4
Min element in tree: [1, Bohemian Rhapsody, Queen, 354, 0, 3] - 1 ticks (0.000001 seconds)
Max element in tree: [10, Shape of You, Ed Sheeran, 233, 0, 2] - 1 ticks (0.000001 seconds)
Element found - 0 ticks (0.000000 seconds)
Removing element in tree: OK - 0 ticks (0.000000 seconds)
Tree size: 9
Tree depth: 4
```

EJERCICIO 3. FILTRAR CANCIONES

El reproductor permitirá ahora filtrar canciones dentro de un rango determinado de identificadores.

Se deberá implementar para ello una función en `bstree.h` que devuelva ***una lista*** con todas las canciones cuyo id esté entre dos valores dados.

PROTOTIPO

```
List *tree_rangeSearch(const BSTree *tree, void *min, void *max);
```

Recuerda que, al igual que en los casos anteriores, será necesaria una rutina auxiliar recursiva para recorrer el árbol a nivel de nodo.

PRUEBA DE FUNCIONAMIENTO BÁSICO

Para probar esta función crea el programa `p4_e2.c` que reciba como argumento el nombre de un fichero de radio, cree un árbol de canciones y llame a la rutina `tree_rangeSearch` pasando como argumentos el árbol y las 2 canciones cuyos ids sean los requeridos. Finalmente, se imprimirá la lista resultante.

Responde en un comentario al final del programa `p4_e2.c` **P2** ¿qué características observas en la lista resultante?, ¿a qué se debe?

A continuación, se muestra el contenido de la salida por consola de la prueba de funcionamiento básico:

Salida esperada:

```
./p4_e2 data_music_10.txt 3 7
```

```
Lista de canciones desde id 3 hasta id 7
[3, Billie Jean, Michael Jackson, 294, 0, 9]
[4, Like a Rolling Stone, Bob Dylan, 369, 0, 7]
[5, Smells Like Teen Spirit, Nirvana, 301, 0, 1]
[6, Hotel California, Eagles, 391, 0, 4]
[7, Sweet Child O' Mine, Guns N' Roses, 356, 0, 0]
```

EJERCICIO 4 (OPCIONAL). FILTRAR CANCIONES

Nuestro reproductor musical permitirá analizar la biblioteca musical. En particular, podrá contar cuántas canciones cumplen una condición. Para ello, se deberá implementar una nueva función que cuente cuántas canciones tienen una duración superior a un valor dado.

PROTOTIPO

```
int tree_countLongSongs(TreeNode *root, int min_duration);
```

Recuerda que, al igual que en los casos anteriores, será necesaria una rutina auxiliar recursiva para recorrer el árbol a nivel de nodo.

PRUEBA DE FUNCIONAMIENTO BÁSICO

Para probar esta función crea el programa `p4_e3.c` que reciba como argumento el nombre de un fichero de radio, cree un árbol de canciones y llame a la rutina `tree_countLongSongs` pasando como argumentos el árbol y la duración mínima.

Responde en un comentario al final del programa `p4_e3.c` **P3.** Tal y como está construido el árbol ¿es posible realizar una poda inteligente para optimizar el recorrido al filtrar por duración?

A continuación, se muestra el contenido del fichero de datos de entrada para la prueba de funcionamiento básico:

```
-PC:~/Documentos/Entregas/solucion$ ./p4_e3 data_music_10.txt 300
Número de canciones con duracion mayor a 300 segundos: 6
```

Antes de dar la práctica por finalizada, recuerda probar varios escenarios de las rutinas del TAD Árbol implementadas. Que todo te salga *purr-fecto.*